

NAFAD PAY

Groupe 4 — Analytics & BI

Data Warehouse · Dashboard BI · Architecture AWS

Documentation Générale Complète du Projet

Implémentation · Résultats · Architecture Cloud · Justifications

Membres

Touba Abdellahi (21003)

Rakiae Dehah (21008)

Khadijetou Mohamed Mewloud (21027)

Oum Elvadhli Cheikh (21032)

Isshagh Alla (24265)

Stack : PostgreSQL 16 · Metabase · Docker · AWS eu-west-3 (Paris)

1. Présentation du Projet

1.1 Contexte

NAFAD PAY est un système fictif de paiement mobile mauritanien. Dans le cadre du module de systèmes distribués et Data Engineering, chaque groupe doit implémenter une couche du système de bout en bout. Le Groupe 4 (G4) est responsable de la couche Analytics & BI : construction du Data Warehouse, chargement des données, et production des dashboards décisionnels pour le Directeur Général.

1.2 Rôle du Groupe 4 dans l'architecture globale

Groupe	Responsabilité
G1 – Core Banking	Base OLTP PostgreSQL (transactions brutes)
G2 – Platform & API	API REST + simulateur de transactions
G3 – Data Engineering	Pipeline ETL Médaillon Bronze/Silver/Gold
G4 – Analytics & BI 	Data Warehouse + Dashboard BI (notre groupe)

Le G4 reçoit les données de staging du G3 et produit les analyses décisionnelles. Un chiffre faux dans notre DWH = une décision stratégique fausse pour le DG.

1.3 Données reçues en entrée

Fichier	Lignes	Description
stg_transactions.csv	100 000	45 colonnes – fichier central
stg_users.csv	10 000	Utilisateurs NAFAD PAY
stg_merchants.csv	500	Marchands – 13 catégories, 15 wilayas
stg_agencies.csv	100	Agences – avec balance float et tier
stg_agents.csv	392	Agents rattachés aux agences
stg_fees.csv	41 391	Frais par transaction
node_metrics.csv	5	Métriques des nœuds serveurs

2. Modélisation du Data Warehouse

2.1 Choix du modèle en étoile (Star Schema)

Nous avons choisi le modèle en étoile (star schema) pour le Data Warehouse. Ce modèle est le standard de l'industrie pour les systèmes analytiques car il optimise les performances des requêtes d'agrégation et simplifie l'écriture des requêtes SQL par les analystes.

2.2 Schéma des tables

Table	Type	Contenu
fact_transactions	Table centrale	100 000 lignes — 1 ligne = 1 transaction
dim_date	Dimension temps	366 lignes — toutes les dates 2024
dim_user	Dimension utilisateur	10 000 lignes — avec SCD Type 2
dim_merchant	Dimension marchand	501 lignes (500 + ligne spéciale -1)
dim_agency	Dimension agence	101 lignes (100 + ligne spéciale -1)
dim_node	Dimension nœud serveur	5 lignes — les 5 nœuds du cluster

2.3 Justification du choix SCD Type 2 pour dim_user

Un utilisateur peut changer de wilaya (région) au cours du temps. Nous avons choisi la stratégie SCD Type 2 (Slowly Changing Dimension) qui crée une nouvelle ligne à chaque changement, avec les colonnes effective_from, effective_to, et is_current.

Stratégie	Mécanisme	Justification
SCD Type 1	Écraser	Simple mais perd l'historique — une transaction de janvier à Nouakchott semblerait venir de Rosso si l'utilisateur a déménagé en mars. Rejeté.
SCD Type 2 	Historiser	Crée une nouvelle ligne à chaque changement. Les analyses géographiques historiques restent correctes. Retenu.
SCD Hybride	Mixte	Type 1 pour les champs peu analytiques, Type 2 pour wilaya. Plus complexe sans valeur ajoutée au MVP.

Colonnes SCD ajoutées dans dim_user :

Colonne	Type	Rôle
effective_from	DATE	Date d'entrée en vigueur de la version
effective_to	DATE	Date de fin — NULL si enregistrement actif
is_current	BOOLEAN	TRUE si version actuelle — index créé dessus

3. Pipeline ETL – Chargement des Données

3.1 Architecture du pipeline

Le pipeline ETL se compose de 9 scripts SQL exécutés dans l'ordre par Docker. Chaque script est idempotent : le relancer deux fois donne le même résultat (propriété garantie par les DROP TABLE IF EXISTS en début de chaque script staging).

Script	Rôle
00_create_staging.sql	Crée les tables temporaires pour recevoir les CSV
01_load_csv.sql	Charge les 4 CSV via COPY PostgreSQL
02_populate_dim_date.sql	Génère toutes les dates 2024 (366 jours)
03_populate_dim_user.sql	Insère 10 000 utilisateurs avec SCD Type 2
04_populate_dim_merchant.sql	Insère 500 marchands
05_populate_dim_agency.sql	Insère 100 agences
07_load_transactions_csv.sql	Vérifie le chargement des 100 000 transactions
08_populate_fact_transactions.sql	INSERT dans fact_transactions avec gestion des anomalies
09_validation_checks.sql	Requêtes de validation post-chargement

3.2 Gestion des anomalies dans fact_transactions

Le script 08_populate_fact_transactions.sql implémente une logique spéciale pour chacune des 4 catégories d'anomalies détectées dans les données :

sync_status	Nb lignes	Part	Traitement
SYNCED	94 923	94.92%	Transaction normale – chargée directement
LAGGING	1 912	1.91%	Retard de sync – include + flag is_lagging = TRUE
PENDING	1 616	1.62%	En cours – include + flag is_pending_sync = TRUE
CONFLICT	1 549	1.55%	Conflit nœuds – logique spéciale (voir section 5)

3.3 Résultats du chargement – validation

366 dim_date	10 000 dim_user	501 dim_merchant	101 dim_agency	5 dim_node	100 000 fact_transactions
------------------------	---------------------------	----------------------------	--------------------------	----------------------	-------------------------------------

FK orphelines : 0 – Toutes les 100 000 transactions ont un utilisateur valide dans dim_user.

4. Dashboard BI – Vue Directeur Général

4.1 Outil choisi : Metabase

Outil	Justification
Metabase 	Open source, gratuit, Docker simple, connecteur PostgreSQL natif, interface intuitive
Amazon QuickSight	18\$/utilisateur/mois – trop coûteux pour le MVP
Looker Studio	Connecteur PostgreSQL parfois payant, dépendance Google
Apache Superset	Plus puissant mais setup complexe pour le MVP

4.2 KPI cards

100 000 Total Transactions	2.09 Mds Volume Total (MRU)	67.3% Taux de Succès
--------------------------------------	---------------------------------------	--------------------------------

4.3 Les 6 visuels du dashboard

#	Visuel	Type	Insight principal
1	Volume mensuel (MRU)	Barres verticales	Volume et nombre de transactions par mois
2	Taux de succès	Donut	SUCCESS 67.3% vs FAILED 32.7%
3	Top 10 Wilayas	Barres horizontales	Classement des wilayas par volume – Nouakchott-Ouest en tête
4	Heures de pointe	Courbe	2 pics : 9h-10h (matin) et 20h-21h (soir)
5	Types de transactions	Barres verticales	TRF > DEP > WIT > PAY > BIL > AIR > SAL > REV
6	Motifs d'échec	Barres horizontales	INSUFFICIENT_BALANCE largement en tête (~25 000)

Volume Total (MRU)

2.1B

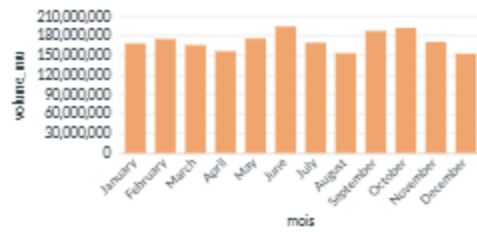
Total Transactions

100,000

Taux de Succès (%)

67.3

Volume mensuel (MRU)

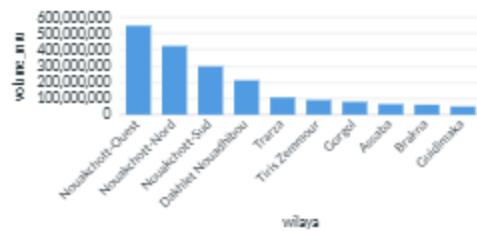


Taux de succès des transactions

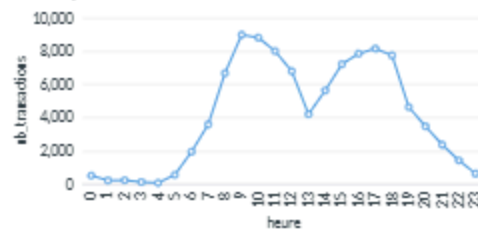
● SUCCESS 67.3%
● FAILED 32.7%



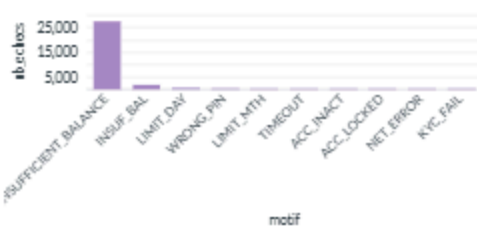
Top 10 Wilayas par volume



Heures de pointe

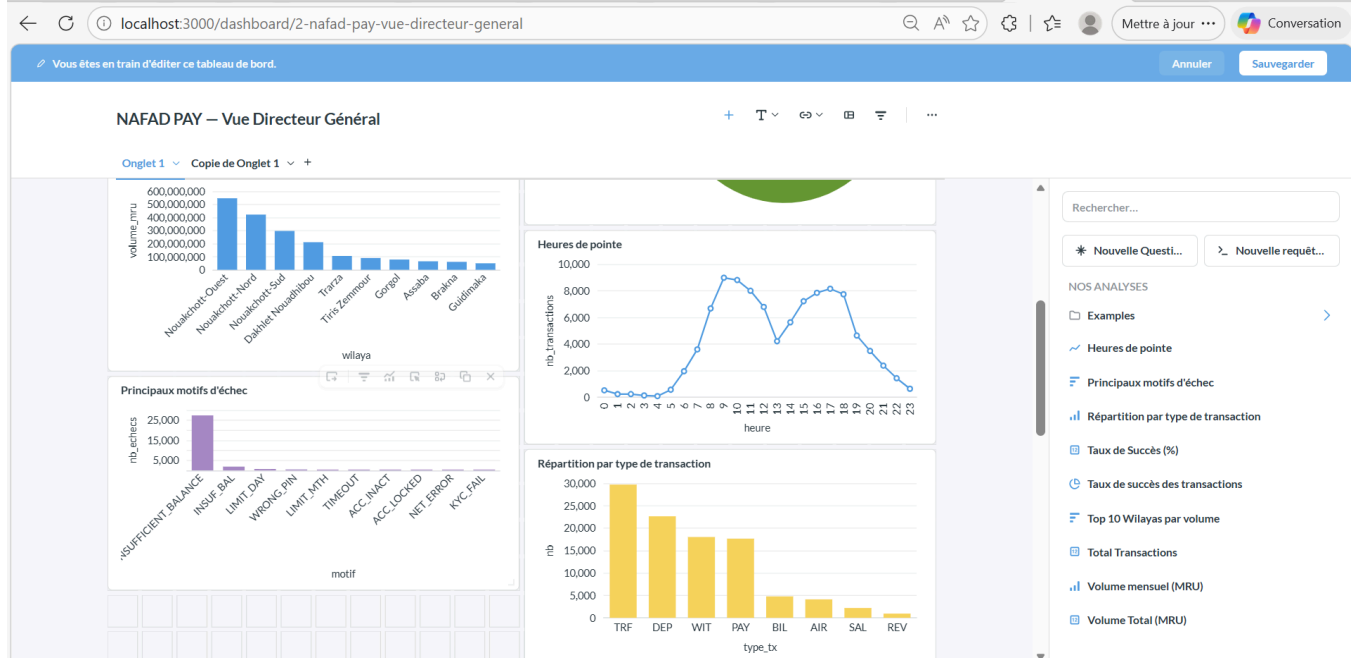
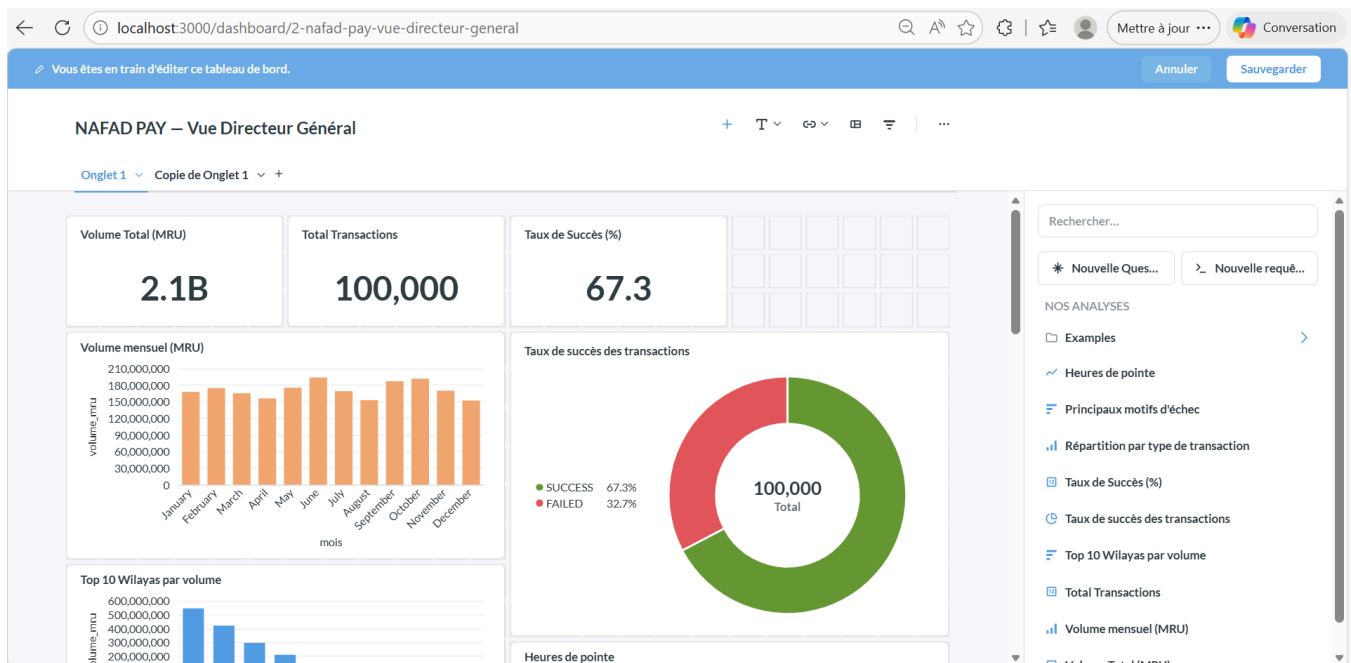


Principaux motifs d'échec



Répartition par type de transaction





4.4 Insights clés détectés

- Heures de pointe : 2 pics à 9h-10h et 20h-21h — le système doit être dimensionné pour ces moments
- Taux d'échec de 32.7% : principalement dû aux soldes insuffisants — axe d'amélioration produit
- Concentration géographique : Nouakchott-Ouest, Nord et Sud dominant le volume
- TRF (transfert) est le type le plus utilisé — service à privilégier dans la roadmap

5. Rapport d'Analyse des Incohérences

5.1 Tableau récapitulatif des anomalies

Anomalie	Volume	Part	Décision
Écart node_metrics vs stg_transactions	1 300 874.55 MRU	0.062%	Documenté — stg_transactions = source de vérité
Transactions CONFLICT	1 549 lignes	1.55%	Inclues avec flag is_conflict = TRUE
CONFLICT amount_node_a/b sans	1 529 lignes	98.7% des CONFLICT	Garder amount (valeur consensus orchestrateur)
CONFLICT avec amount_node_a	20 lignes	1.3% des CONFLICT	Prendre amount_node_a (nœud primaire)
Transactions LAGGING	1 912 lignes	1.91%	Inclues avec flag is_lagging = TRUE
Transactions PENDING	1 616 lignes	1.62%	Inclues avec flag is_pending_sync = TRUE

5.2 Q1 — Écart entre node_metrics et stg_transactions

Total stg_transactions SUM(amount) : 2 089 707 250.00 MRU
Total node_metrics SUM(total_amount) : 2 091 008 124.55 MRU
Écart : 1 300 874.55 MRU (0.0623%)

Cet écart de 0.062% s'explique par 3 facteurs dans un système distribué :

1. Transactions PENDING (1 616 lignes) : non encore finalisées au moment de la capture des métriques
2. Clock skew : les métriques sont capturées à des moments légèrement différents selon l'horloge de chaque serveur
3. Arrondi numérique : les totaux de node_metrics.csv sont des flottants (ex: 4.349257e+08), introduisant des erreurs cumulées

Décision : utiliser stg_transactions comme source de vérité pour le DWH. C'est la table transactionnelle complète et cohérente.

5.3 Q2 & Q3 — Stratégie CONFLICT

1 549 transactions ont sync_status = CONFLICT, signifiant que les deux nœuds n'étaient pas d'accord sur le montant. Parmi elles, seulement 20 ont amount_node_a ou amount_node_b renseignés.

Cas	Décision
~20 CONFLICT avec amount_node_a renseigné	Prendre amount_node_a (nœud primaire = source de vérité par convention)
~1 529 CONFLICT sans amount_node_a ni amount_node_b	Garder amount (valeur de consensus de l'orchestrateur, seule disponible)
Tous les CONFLICT (1 549)	Flagger is_conflict = TRUE pour alerte dans le dashboard

Pourquoi ne pas exclure les CONFLICT ? Exclure 1 549 lignes = perdre 1.55% des transactions et biaiser tous les agrégats (volume total, taux de succès, analyses par wilaya).

5.4 Q4 – Clock skew : last_synced_at antérieur à created_at

Dans un système distribué, chaque serveur a sa propre horloge. Sans synchronisation NTP stricte, les horloges divergent de quelques secondes à quelques minutes. Ici, le nœud qui enregistre last_synced_at avait son horloge en avance sur le nœud qui a enregistré created_at.

Impact sur les agrégations : si on utilise last_synced_at pour des fenêtres temporelles, des transactions du 15 janvier peuvent apparaître comme synchronisées le 14 janvier, faussant les comptages journaliers.

Décision : utiliser uniquement transaction_date (basée sur created_at) pour toutes les analyses temporelles dans le DWH. last_synced_at est conservé pour audit uniquement.

5.5 Q6 – Temps réel ET comparaisons mois vs mois

Approche	Analyse
Architecture Lambda	Couche batch (Redshift) + couche streaming (Kinesis). Deux systèmes à maintenir, cohérence complexe, coût élevé. Rejetée au MVP.
Architecture Kappa	Tout par le streaming, historique recalculé. Coûteux pour rejouer 12 mois de données. Rejetée.
Materialized Views 	Redshift MV rafraîchies toutes les heures. Une seule source de vérité, coût minimal, opération simple. Retenu pour l'At Scale.

6. Infrastructure Docker – Reproductibilité

6.1 Architecture Docker Compose

Service	Image	Rôle
postgres_dwh	postgres:16-alpine	Data Warehouse PostgreSQL – port 5432
etl	postgres:16-alpine	Runner ETL – exécute les 9 scripts SQL en séquence
metabase	metabase/metabase:latest	Dashboard BI – port 3000

6.2 Commandes de démarrage

```
# 1. Configuration
cp .env.example .env
# 2. Démarrer PostgreSQL
docker-compose up -d postgres_dwh
```

```
# 3. Charger les données
docker-compose --profile etl run --rm etl
# 4. Démarrer le dashboard
docker-compose up -d metabase
# 5. Ouvrir http://localhost:3000
```

6.3 Structure du dépôt GitHub

Fichier	Auteur	Contenu
schema/ddl_star_schema.sql	Ishaghe	DDL des 6 tables du star schema
etl/00 à 05_*.sql	Oumlvadli	Scripts ETL des dimensions
etl/07 à 09_*.sql	Yemame	ETL fact_transactions + validation
bi/queries_dashboard.sql	Khadije	6 requêtes analytiques du dashboard
scripts/run_all.sh	Touba	Script de démarrage automatique
docker-compose.yml	Touba	Orchestration des 3 services Docker
docs/archi_early_stage.md	Ishaghe	Architecture AWS MVP
docs/archi_at_scale.md	Oumlvadli + Khadije + Yemame	Architecture AWS cible
docs/rapport_analyse.md	Yemame	Rapport sur les incohérences
README.md	Touba	Guide de démarrage en 4 étapes

7. Architecture AWS – Early Stage (MVP)

7.1 Pourquoi eu-west-3 Paris ?

Région	Distance Nouakchott	Latence estimée	Décision
eu-west-3 Paris 	~4 000 km	40-60 ms	Référence – RETENU
af-south-1 Le Cap	~7 000 km	120-150 ms	Plus loin, moins de services
eu-west-1 Dublin	~4 300 km	50-70 ms	Alternative acceptable
Hetzner fsn1 Falkenstein	~4 500 km	50-70 ms	Moins cher, pas de RDS managé

7.2 Composants du MVP

Service	Instance/Config	Rôle
RDS PostgreSQL 16	db.t3.medium	Data Warehouse – subnet privé – backup 7j
EC2 t3.medium	Amazon Linux 2023	Metabase – subnet public – SSM Session Manager
S3 Bucket	Standard	Stockage CSV staging

Secrets Manager	1 secret	Mot de passe RDS – rotation manuelle
VPC	10.0.0.0/16	Réseau isolé – 1 subnet public + 1 subnet privé

7.3 ADR-lite – Décisions techniques

Décision	Choix retenu	Justification
PostgreSQL RDS vs Redshift	RDS PostgreSQL 16 retenu	Volume < 1M lignes. Redshift est surdimensionné et 3× plus cher pour ce volume.
Metabase vs QuickSight	Metabase retenu	Open source, gratuit. QuickSight coûte 18\$/utilisateur/mois.
ETL SQL vs dbt	Scripts SQL retenus	100K lignes se chargent en < 5 min. dbt ajoute de la complexité sans valeur au MVP.
eu-west-3 vs af-south-1	eu-west-3 retenu	Plus proche de Nouakchott (4 000 km vs 7 000 km). Plus fourni en services managés.

7.4 Estimation de coût mensuel

Service	Config	Coût/mois
RDS PostgreSQL 16	db.t3.medium	~55 \$
EC2 Metabase	t3.medium	~30 \$
EBS stockage	gp3 80 Go	~6 \$
S3 staging	10 Go	~0.23 \$
Secrets Manager	1 secret	~0.40 \$
Data transfer	~10 Go/mois	~0.90 \$
TOTAL		~93 \$/mois

8. Architecture AWS – At Scale

8.1 Cible : 5M transactions/mois, 500K utilisateurs

Contrainte	Valeur
Volume	5M transactions/mois, 500K utilisateurs actifs
QPS en pic	> 500 requêtes analytiques/seconde
SLA	Disponibilité 99.9%, RPO < 1h, RTO < 4h
Latence dashboard	p99 < 1 seconde
Sécurité PII	phone, nni, email – masquage obligatoire
Compliance	Row-Level Security par wilaya

8.2 Composants At Scale

Service	Rôle
Redshift Serverless	DWH analytique – scale auto – pause auto hors heures
ECS Fargate	Metabase – auto-scaling 1 à 5 tâches
ALB + WAF	Point d'entrée – OWASP Top 10 – rate limiting
Cognito SSO	Authentification SAML/OIDC – session 8h max – MFA admins
ElastiCache Redis	Cache requêtes fréquentes – TTL 1h KPI – 5min alertes
Glue + Macie	Catalogue données + détection automatique PII
S3 Parquet Gold	Stockage analytique – partitionné par transaction_date
dbt Core + CI/CD	Transformations testées – GitHub Actions → CodePipeline

8.3 Sécurité & gouvernance PII

Mesure	Détail
Row-Level Security	Un chef d'agence ne voit que les transactions de sa wilaya via Redshift RLS
Column Masking	phone, nni, email masqués pour les analystes juniors – Dynamic Data Masking Redshift
IAM Least Privilege	Chaque service a un rôle IAM minimal – jamais de compte partagé
Audit complet	CloudTrail + Redshift audit logs → S3 → requêteable via Athena
Chiffrement	At-rest : KMS sur Redshift + S3 SSE-KMS – In-transit : TLS 1.2+ obligatoire

8.4 Threat model – Top 3 attaques

#	Attaque	Mitigation
1	Analyste exfiltre un dump	Quota export 10K lignes + audit CloudTrail + alerte CloudWatch si volume > seuil
2	Token Metabase compromis	Cognito SSO + session 8h max + rotation automatique des credentials
3	Requête runaway 500\$	Redshift Query Monitoring Rules + alarme AWS Budget + concurrency scaling limit

8.5 Estimation de coût mensuel At Scale

Service	Config	Coût/mois
Redshift Serverless	~8h/jour	~120 \$
ECS Fargate	1-5 tâches	~40 \$

ALB	1 load balancer	~18 \$
ElastiCache Redis	t3.micro	~15 \$
S3 Parquet	1 To	~23 \$
Glue + Macie	~10 runs/jour	~15 \$
Cognito	10K users	~0 \$ (free tier)
CloudTrail + CloudWatch	Tous les logs	~10 \$
TOTAL		~241 \$/mois

9. Comparaison Early Stage vs At Scale

Critère	Early Stage (MVP)	At Scale
Coût mensuel	~93 \$	~241 \$
Volume max supporté	< 5M lignes	1Md+ lignes
Utilisateurs BI simultanés	5-10	100+
Disponibilité	99%	99.9%
Authentification	Login/password Metabase	SSO Cognito SAML/OIDC
Sécurité PII	Basique (TLS)	RLS + Column Masking + Macie
ETL	Scripts SQL manuels	dbt Core + CI/CD
Stockage DWH	RDS PostgreSQL gp3	Redshift Serverless + S3 Parquet
Cache	Aucun	ElastiCache Redis (TTL 1h)
Temps réel	Non	Materialized Views refresh 1h
Latence p99 dashboard	< 3 secondes	< 1 seconde

10. Conclusion

Le Groupe 4 a livré une implémentation complète et fonctionnelle de la couche Analytics & BI de NAFAD PAY. Le projet couvre l'intégralité de la chaîne : de la réception des données brutes jusqu'aux dashboards décisionnels, en passant par la modélisation du Data Warehouse et le déploiement sur AWS.

10.1 Synthèse des livrables

Livable	Poids	Contenu
Implémentation (DDL + ETL + Dashboard)	40%	Star schema 6 tables, 9 scripts ETL, 100K tx chargées, dashboard Metabase 6 visuels + 3 KPI
Architecture At Scale	20%	Redshift Serverless, SSO Cognito, RLS, Macie, dbt CI/CD, threat model, plan de migration

Architecture Early Stage	15%	RDS PostgreSQL, EC2 Metabase, VPC, 4 ADR-lite, justification eu-west-3
Questions d'investigation	15%	6 questions répondues : clock skew, CONFLICT, SCD, Materialized Views, écart node_metrics
Qualité code / README	10%	Scripts idempotents, validation post-ETL, README 4 étapes, run_all.sh

10.2 Points forts du projet

- 100% reproductible : docker-compose up + run_all.sh suffit à tout démarrer
- Idempotence garantie : les scripts ETL peuvent être rejoués sans effets de bord
- Anomalies documentées et gérées : CONFLICT, LAGGING, PENDING, clock skew
- Architecture justifiée : chaque choix AWS est argumenté avec des ADR-lite
- Dashboard lisible : 6 visuels cohérents avec des insights actionnables pour le DG

Le projet est disponible sur GitHub avec un README complet. Commande unique de démarrage :
./scripts/run_all.sh — puis ouvrir <http://localhost:3000>